

In []:

Signals and Systems

Module 1

Signals and Systems

Introduction

Signals are mathematical functions used to represent physical phenomena that vary with one or more independent variables such as time or space. Systems process these signals to produce desired outputs. Understanding the classification of signals, basic signal operations, elementary signals, and system properties is fundamental to the analysis and design of engineering systems. This notebook demonstrates these concepts using Python with graphical visualization.

1.1 Definition of a Signal and a System

Aim

- To understand the concepts of signals and systems.
- To generate and visualize basic signals using Python.
- To observe the response of a simple system.

Problem 1

Problem Statement

Generate a continuous-time sinusoidal signal

$$x(t) = \sin(2\pi t)$$

for the interval

$$0 \leq t \leq 2$$

and plot the signal.

```
In [1]: import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(0,2,1000)

x = np.sin(2*np.pi*t)

plt.figure(figsize=(10,4))

plt.plot(t,x,color='blue')

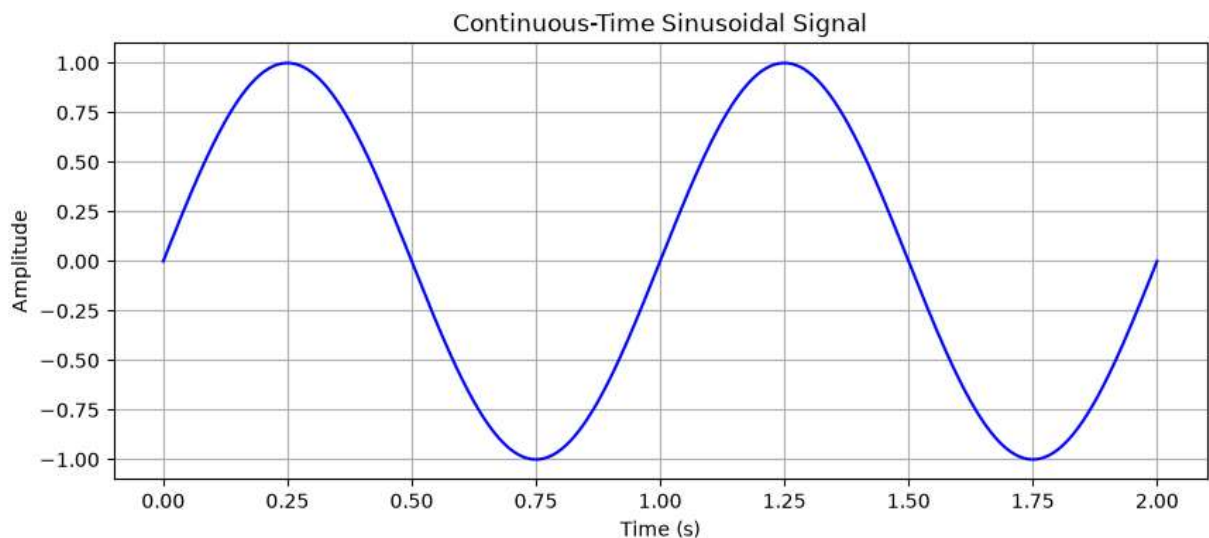
plt.title("Continuous-Time Sinusoidal Signal")

plt.xlabel("Time (s)")

plt.ylabel("Amplitude")

plt.grid(True)

plt.show()
```



Observation

- A sinusoidal signal varies continuously with time.
- The signal repeats periodically with constant amplitude.
- Continuous-time signals are represented over a continuous range of time.

Problem 2

Problem Statement

Consider a simple system

$$y(t) = 2x(t)$$

where

$$x(t) = \sin(2\pi t)$$

Plot both the input and output signals on the same graph.

```
In [2]: import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(0,2,1000)

x = np.sin(2*np.pi*t)

y = 2*x

plt.figure(figsize=(10,4))

plt.plot(t,x,label="Input Signal")

plt.plot(t,y,label="Output Signal")

plt.title("Input and Output of a Simple System")

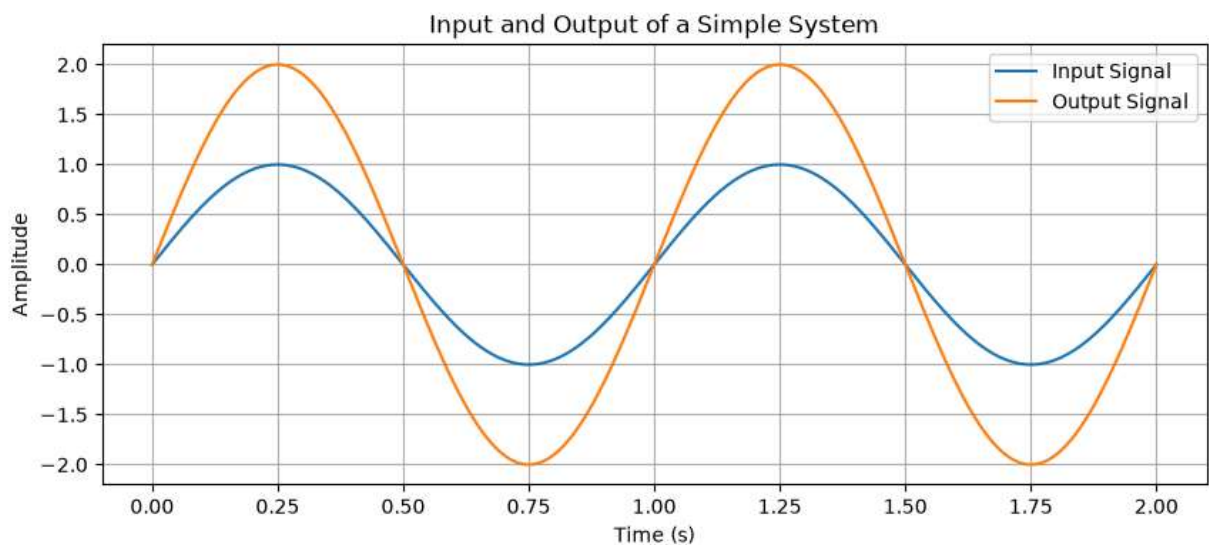
plt.xlabel("Time (s)")

plt.ylabel("Amplitude")

plt.legend()

plt.grid(True)

plt.show()
```



Observation

- The system amplifies the input signal by a factor of two.
- The output has the same frequency as the input.

- Only the amplitude changes.

1.2 Classification of Signals

Aim

- To classify signals based on their characteristics.
- To distinguish between different signal types using graphical visualization.

Problem 3

Problem Statement

Generate and compare an even signal

$$x(t) = \cos(t)$$

and an odd signal

$$x(t) = \sin(t)$$

on the same graph.

```
In [3]: import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(-2*np.pi,2*np.pi,1000)

even = np.cos(t)

odd = np.sin(t)

plt.figure(figsize=(10,4))

plt.plot(t,even,label="Even Signal : cos(t)")

plt.plot(t,odd,label="Odd Signal : sin(t)")

plt.title("Even and Odd Signals")

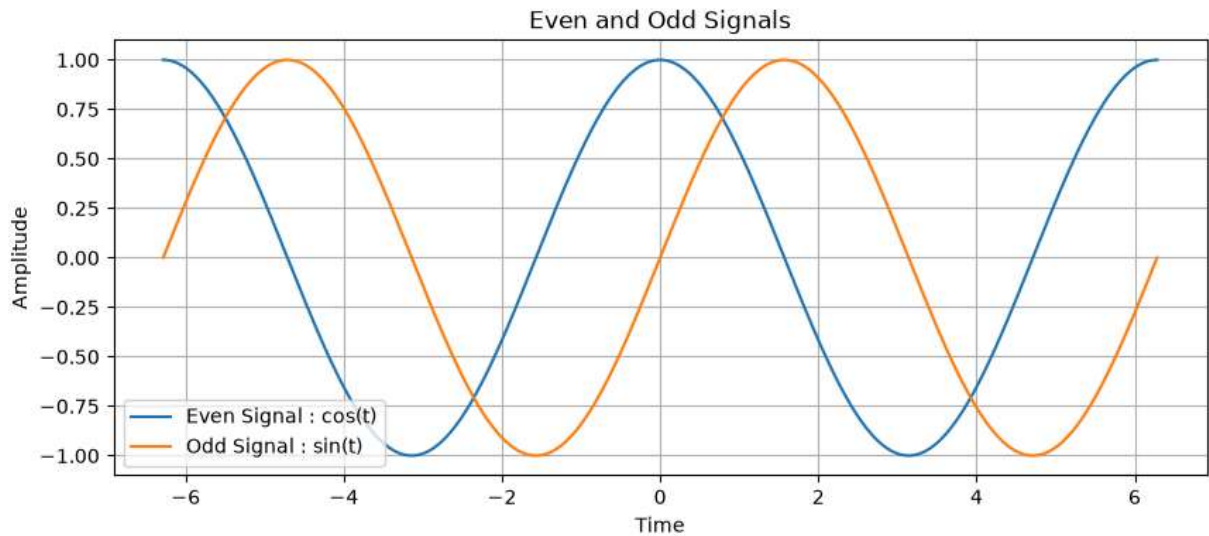
plt.xlabel("Time")

plt.ylabel("Amplitude")

plt.legend()

plt.grid(True)

plt.show()
```



Observation

- The cosine signal is symmetric about the vertical axis and is therefore an even signal.
- The sine signal is symmetric about the origin and is therefore an odd signal.

Problem 4

Problem Statement

Generate a deterministic signal

$$x(t) = \sin(2\pi t)$$

and compare it with a random signal generated using Python.

```
In [5]: import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(0,2,1000)

deterministic = np.sin(2*np.pi*t)

random_signal = np.random.randn(1000)

plt.figure(figsize=(10,5))

plt.subplot(2,1,1)

plt.plot(t,deterministic)

plt.title("Deterministic Signal")

plt.grid(True)

plt.subplot(2,1,2)
```

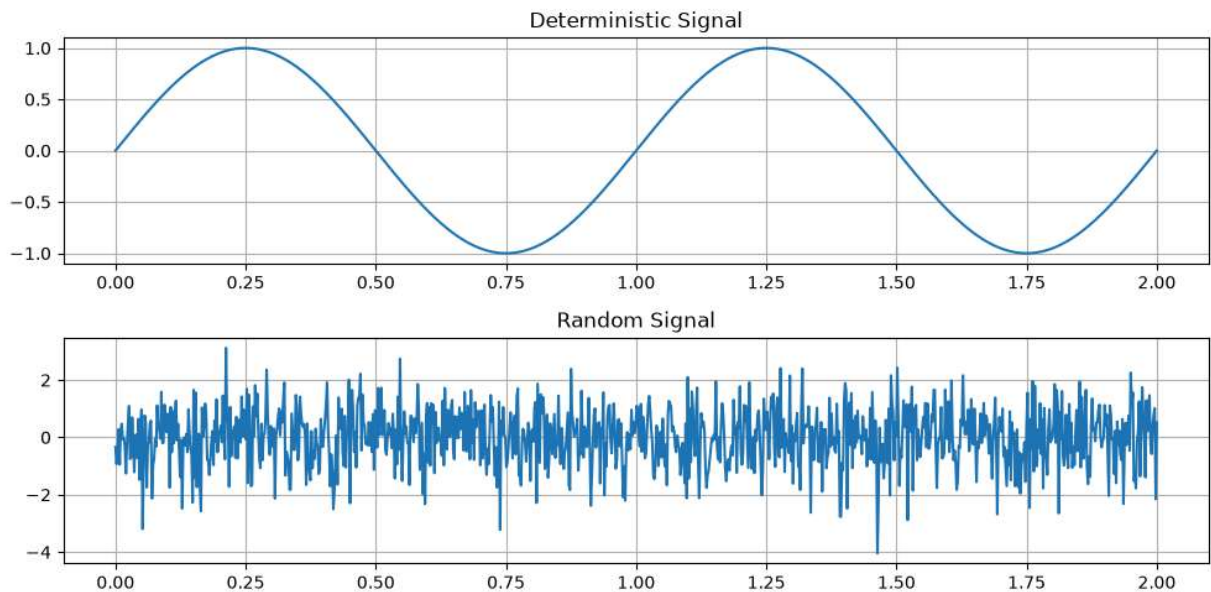
```
plt.plot(t, random_signal)

plt.title("Random Signal")

plt.grid(True)

plt.tight_layout()

plt.show()
```



Observation

- A deterministic signal can be described using a mathematical equation.
- A random signal varies unpredictably and cannot be expressed by a fixed mathematical expression.

Conclusion

In this section, the concepts of signals and systems were introduced. Continuous-time signals were generated and processed through a simple system. Different classes of signals, including even, odd, deterministic, and random signals, were visualized and compared using Python.

1.2.1 Continuous-Time vs. Discrete-Time Signals

Aim

- To distinguish between continuous-time and discrete-time signals.
- To visualize the graphical representation of both signal types.
- To compare their characteristics.

Problem 5

Problem Statement

Generate the continuous-time signal

$$x(t) = \sin(2\pi t)$$

and the discrete-time signal

$$x[n] = \sin\left(\frac{\pi n}{4}\right)$$

Plot both signals for comparison.

```
In [6]: import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(0,2,1000)
n = np.arange(0,17)

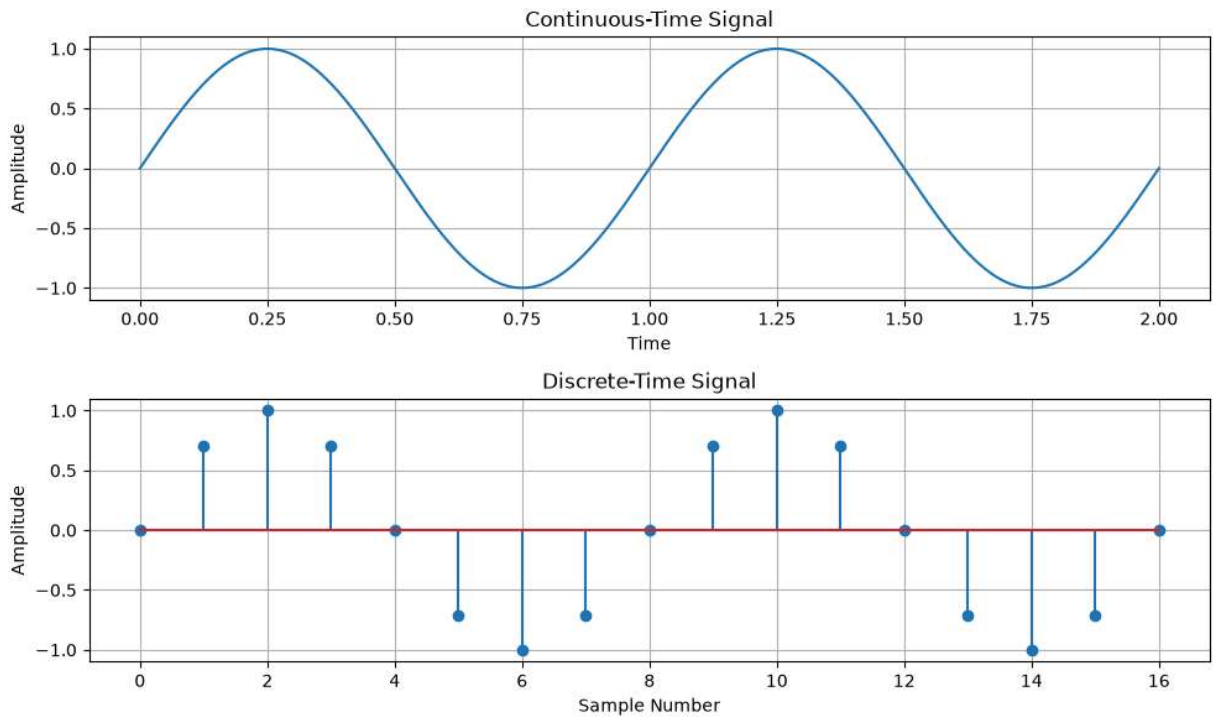
x_ct = np.sin(2*np.pi*t)
x_dt = np.sin(np.pi*n/4)

plt.figure(figsize=(10,6))

plt.subplot(2,1,1)
plt.plot(t,x_ct)
plt.title("Continuous-Time Signal")
plt.xlabel("Time")
plt.ylabel("Amplitude")
plt.grid(True)

plt.subplot(2,1,2)
plt.stem(n,x_dt)
plt.title("Discrete-Time Signal")
plt.xlabel("Sample Number")
plt.ylabel("Amplitude")
plt.grid(True)

plt.tight_layout()
plt.show()
```



Observation

- Continuous-time signals exist for every instant of time.
- Discrete-time signals are defined only at specific sampling instants.
- The continuous signal appears as a smooth curve, whereas the discrete signal consists of isolated samples.

Problem 6

Problem Statement

Generate a continuous-time exponential signal

$$x(t) = e^{-t}$$

and obtain its sampled discrete-time version.

Compare both graphically.

```
In [7]: import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(0,5,1000)
n = np.arange(0,21)

x_ct = np.exp(-t)
x_dt = np.exp(-0.25*n)

plt.figure(figsize=(10,6))
```

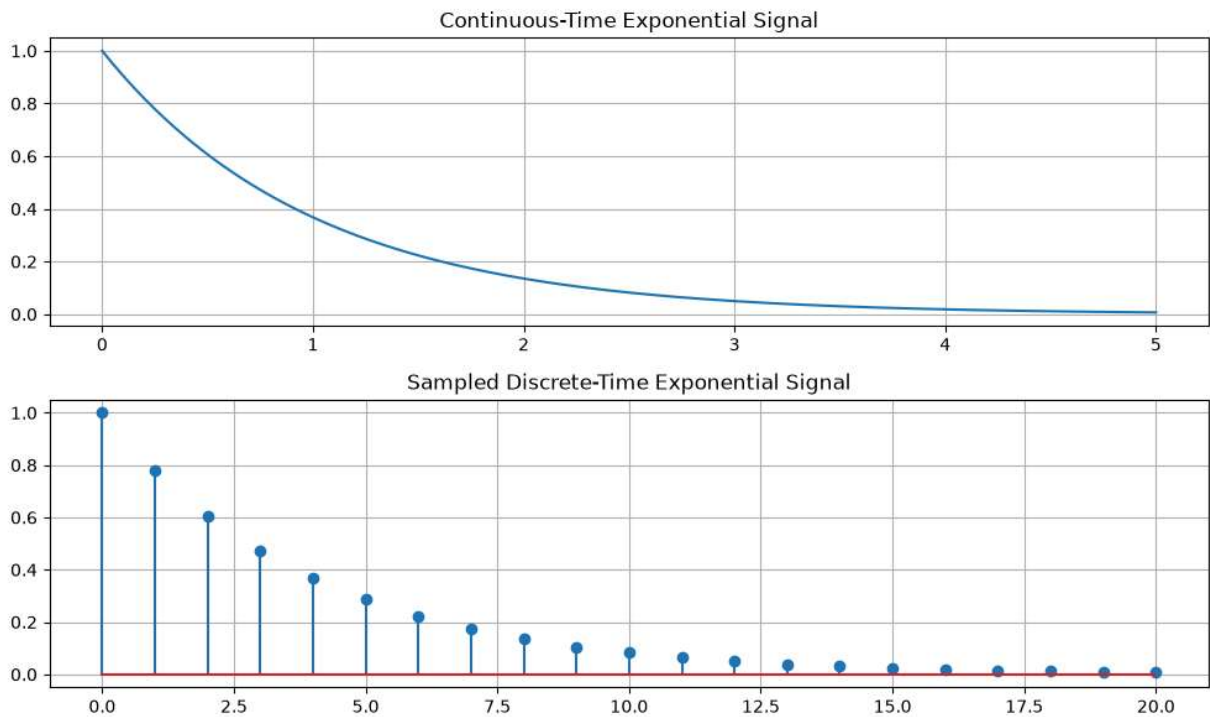


```
plt.subplot(2,1,1)
plt.plot(t,x_ct)
plt.title("Continuous-Time Exponential Signal")
plt.grid(True)

plt.subplot(2,1,2)
plt.stem(n,x_dt)
plt.title("Sampled Discrete-Time Exponential Signal")
plt.grid(True)

plt.tight_layout()

plt.show()
```



Observation

- The discrete-time signal represents sampled values of the continuous-time signal.
- Increasing the sampling rate improves the approximation of the continuous-time signal.

1.2.2 Periodic, Aperiodic, Energy and Power Signals

Aim

- To distinguish between periodic and aperiodic signals.
- To calculate the energy and power of signals using Python.

Problem 7

Problem Statement

Generate the periodic signal

$$x(t) = \sin(2\pi t)$$

and the aperiodic signal

$$x(t) = e^{-t}$$

Plot both signals and compare their characteristics.

```
In [8]: import numpy as np
import matplotlib.pyplot as plt

t=np.linspace(0,5,1000)

periodic=np.sin(2*np.pi*t)

aperiodic=np.exp(-t)

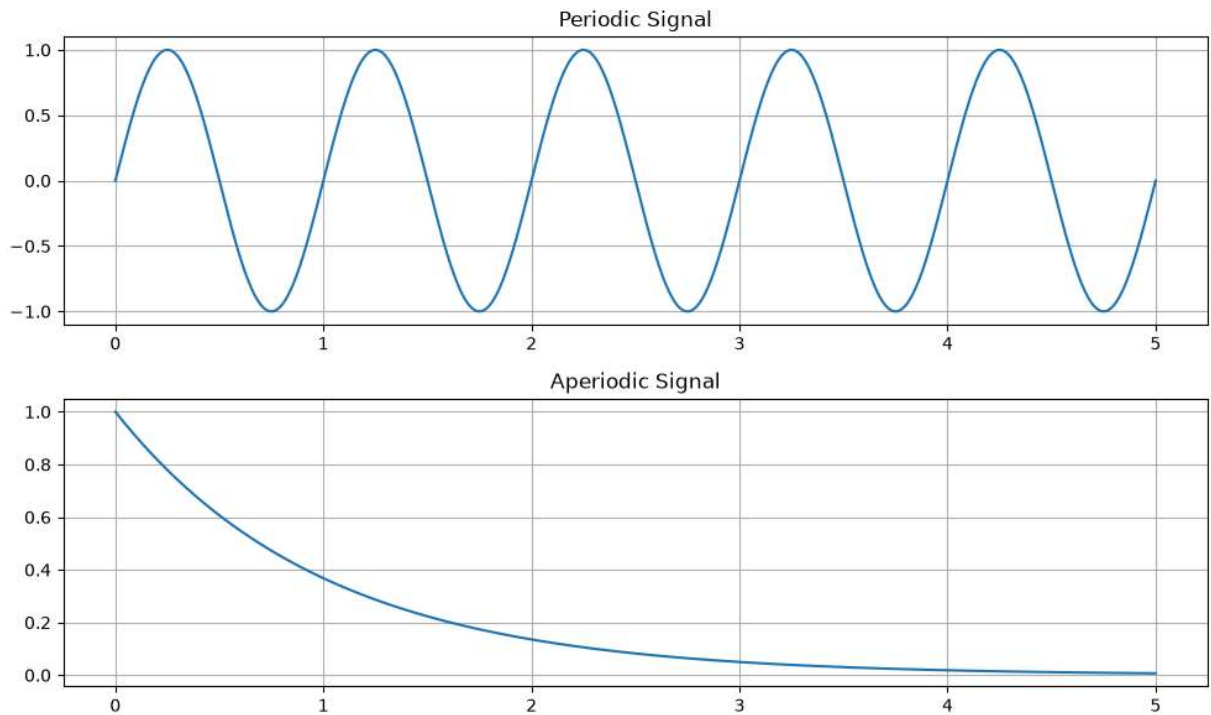
plt.figure(figsize=(10,6))

plt.subplot(2,1,1)
plt.plot(t,periodic)
plt.title("Periodic Signal")
plt.grid(True)

plt.subplot(2,1,2)
plt.plot(t,aperiodic)
plt.title("Aperiodic Signal")
plt.grid(True)

plt.tight_layout()

plt.show()
```



Observation

- A periodic signal repeats itself after a fixed interval.
- An aperiodic signal does not repeat with time.

Problem 8

Problem Statement

Calculate the energy of the finite-duration signal

$$x(t) = e^{-t}$$

over the interval

$$0 \leq t \leq 5$$

and display the result.

```
In [9]: import numpy as np
import matplotlib.pyplot as plt

t=np.linspace(0,5,1000)

x=np.exp(-t)

energy=np.trapezoid(x**2,t)

plt.figure(figsize=(9,4))
```

```
plt.plot(t,x)

plt.title("Energy Signal")

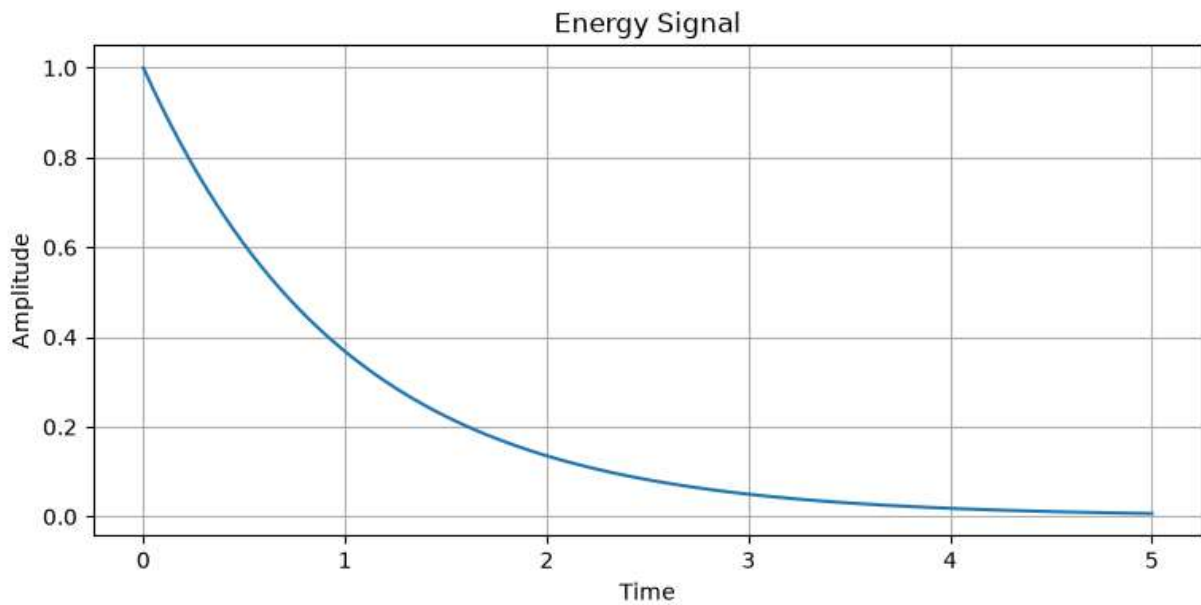
plt.xlabel("Time")

plt.ylabel("Amplitude")

plt.grid(True)

plt.show()

print("Signal Energy =",energy)
```



Signal Energy = 0.4999814748511182

Observation

- The signal possesses finite energy.
- Therefore, it is classified as an energy signal.

Problem 9

Problem Statement

Determine the average power of the periodic signal

$$x(t) = \sin(2\pi t)$$

over one complete period.

```
In [10]: import numpy as np
import matplotlib.pyplot as plt
```

```

t=np.linspace(0,1,1000)
x=np.sin(2*np.pi*t)
power=np.trapezoid(x**2,t)/(t[-1]-t[0])

plt.figure(figsize=(9,4))

plt.plot(t,x)

plt.title("Power Signal")

plt.xlabel("Time")

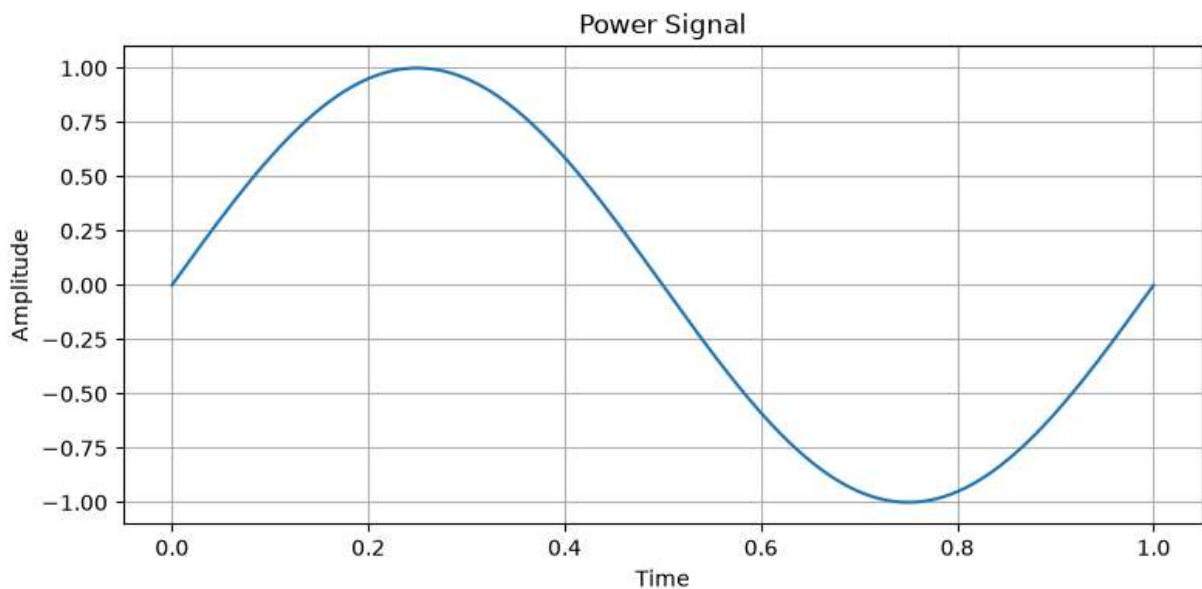
plt.ylabel("Amplitude")

plt.grid(True)

plt.show()

print("Average Power =",power)

```



Average Power = 0.5

Observation

- The average power of the periodic signal is finite.
- Hence, the signal is classified as a power signal.

Conclusion

In this section, continuous-time and discrete-time signals were compared graphically. The characteristics of periodic and aperiodic signals were examined. The energy of a finite-duration signal and the average power of a periodic signal were calculated using numerical integration, illustrating the distinction between energy and power signals.

1.3 Basic Operations on Signals

Aim

- To understand basic signal operations.
- To perform amplitude scaling, time shifting, and time reversal.
- To compare the original and transformed signals graphically.

Problem 10

Problem Statement

Generate the signal

$$x(t) = \sin(t)$$

Plot the following on the same graph:

- Original Signal
- Amplitude Scaled Signal ($2x(t)$)
- Time-Shifted Signal ($x(t-1)$)

```
In [11]: import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(-5,5,1000)

x = np.sin(t)

scaled = 2*x

shifted = np.sin(t-1)

plt.figure(figsize=(10,5))

plt.plot(t,x,label="Original Signal")

plt.plot(t,scaled,label="Amplitude Scaled : 2x(t)")

plt.plot(t,shifted,label="Time Shifted : x(t-1)")

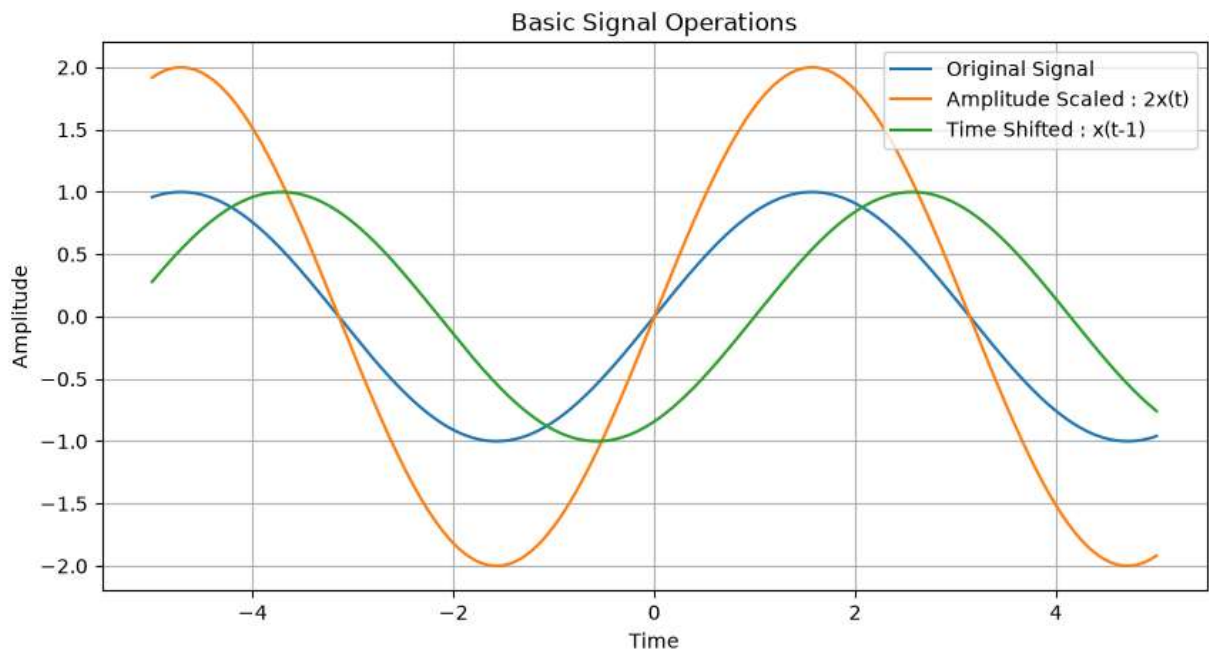
plt.title("Basic Signal Operations")

plt.xlabel("Time")

plt.ylabel("Amplitude")

plt.grid(True)
```

```
plt.legend()
plt.show()
```



Observation

- Amplitude scaling changes only the magnitude of the signal.
- Time shifting delays the signal without changing its shape.
- The waveform remains unchanged except for its position and amplitude.

Problem 11

Problem Statement

Generate the signal

$$x(t) = \cos(t)$$

Plot the original signal and its time-reversed version

$$x(-t)$$

to illustrate the effect of time reversal.

```
In [12]: import numpy as np
import matplotlib.pyplot as plt

t=np.linspace(-5,5,1000)

x=np.cos(t)
```

```
reverse=np.cos(-t)

plt.figure(figsize=(10,5))

plt.plot(t,x,label="Original Signal")

plt.plot(t,reverse,label="Time Reversed Signal")

plt.title("Time Reversal")

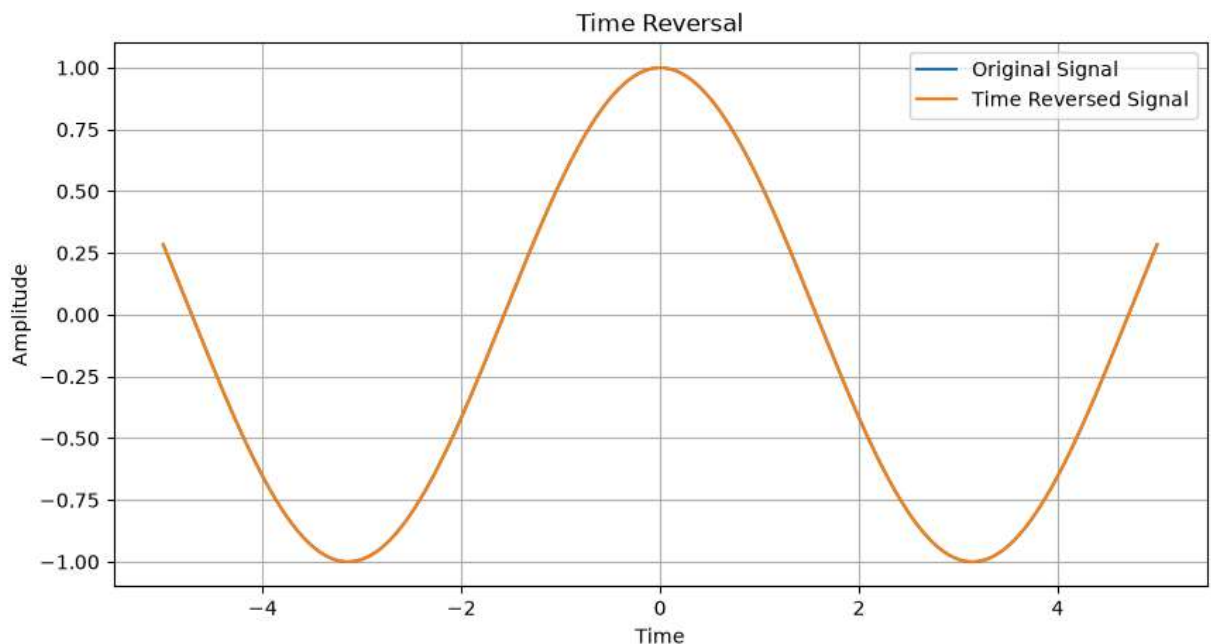
plt.xlabel("Time")

plt.ylabel("Amplitude")

plt.grid(True)

plt.legend()

plt.show()
```



Observation

- Time reversal reflects the signal about the vertical axis.
- Since the cosine function is an even function, the original and time-reversed signals overlap.

1.4 Elementary Signals

Aim

- To generate elementary signals.

- To study their graphical characteristics.
- To compare unit step, ramp, impulse, and sinusoidal signals.

Problem 12

Problem Statement

Generate the following elementary signals over the interval

$$-2 \leq t \leq 2$$

- Unit Step Signal
- Ramp Signal
- Unit Impulse (Approximated)

```
In [13]: import numpy as np
import matplotlib.pyplot as plt

t=np.linspace(-2,2,1000)

step=np.heaviside(t,1)

ramp=np.maximum(t,0)

impulse=np.zeros_like(t)
impulse[np.argmin(np.abs(t))]=1

plt.figure(figsize=(10,8))

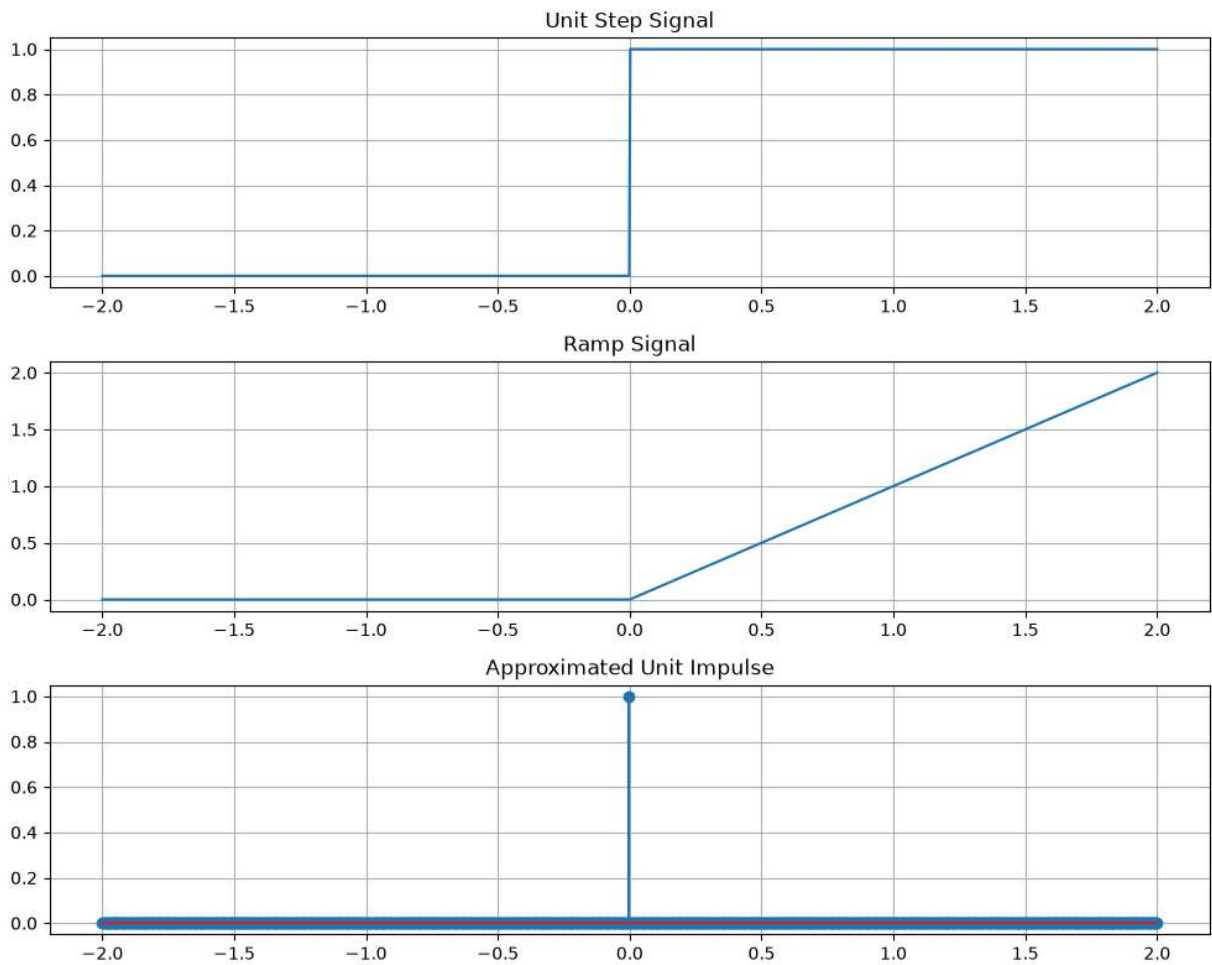
plt.subplot(3,1,1)
plt.plot(t,step)
plt.title("Unit Step Signal")
plt.grid(True)

plt.subplot(3,1,2)
plt.plot(t,ramp)
plt.title("Ramp Signal")
plt.grid(True)

plt.subplot(3,1,3)
plt.stem(t,impulse)
plt.title("Approximated Unit Impulse")
plt.grid(True)

plt.tight_layout()

plt.show()
```



Observation

- The unit step signal changes from 0 to 1 at ($t=0$).
- The ramp signal increases linearly for positive time.
- The impulse signal exists only at one instant.

Problem 13

Problem Statement

Generate and compare the following sinusoidal signals:

$$x_1(t) = \sin(2\pi t)$$

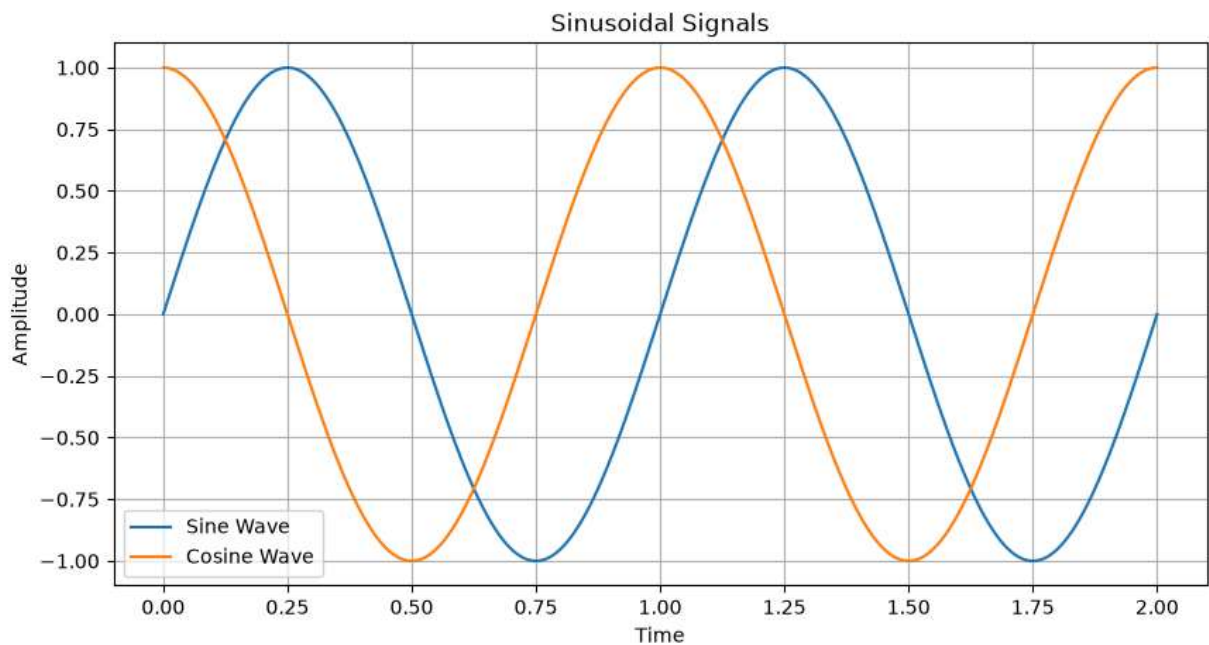
$$x_2(t) = \cos(2\pi t)$$

Plot both signals on the same graph.

```
In [14]: import numpy as np
import matplotlib.pyplot as plt

t=np.linspace(0,2,1000)
```

```
sin_wave=np.sin(2*np.pi*t)
cos_wave=np.cos(2*np.pi*t)
plt.figure(figsize=(10,5))
plt.plot(t,sin_wave,label="Sine Wave")
plt.plot(t,cos_wave,label="Cosine Wave")
plt.title("Sinusoidal Signals")
plt.xlabel("Time")
plt.ylabel("Amplitude")
plt.legend()
plt.grid(True)
plt.show()
```



Observation

- Both sine and cosine waves are periodic signals.
- They have the same frequency and amplitude.
- The cosine wave leads the sine wave by 90° .

1.5 Properties of Systems (Linearity, Time-Invariance, Causality)

Aim

- To study the important properties of systems.
- To verify linearity, time-invariance, and causality using Python.
- To understand the significance of these properties in signal processing.

Problem 14

Problem Statement

Consider the system

$$y(t) = 2x(t)$$

where

$$x(t) = \sin(t)$$

Plot the input and output signals and verify whether the system is:

- Linear
- Time-Invariant
- Causal

```
In [15]: import numpy as np
import matplotlib.pyplot as plt

# Time axis
t = np.linspace(-5,5,1000)

# Input signal
x = np.sin(t)

# Output signal
y = 2*x

plt.figure(figsize=(10,5))

plt.plot(t,x,label="Input Signal x(t)")

plt.plot(t,y,label="Output Signal y(t)=2x(t)")

plt.title("Verification of System Properties")

plt.xlabel("Time")

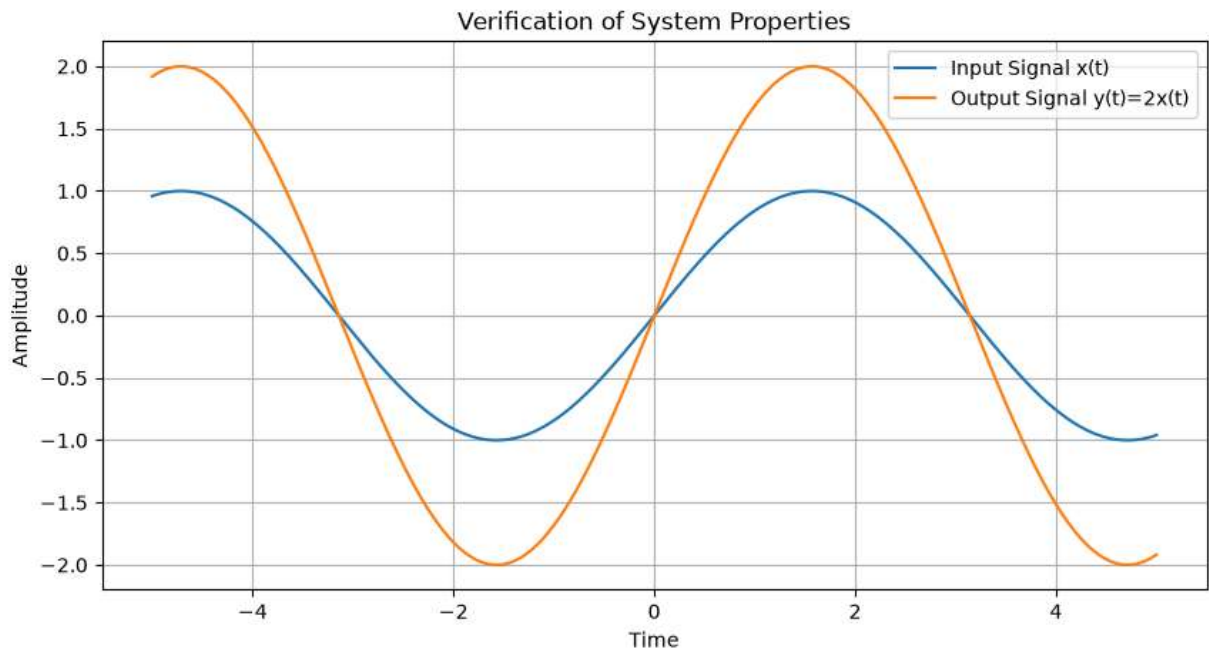
plt.ylabel("Amplitude")

plt.grid(True)

plt.legend()
```

```
plt.show()

print("System Analysis")
print("-----")
print("Linearity      : Satisfied")
print("Time Invariance: Satisfied")
print("Causality       : Satisfied")
```



System Analysis

```
-----
Linearity      : Satisfied
Time Invariance: Satisfied
Causality      : Satisfied
```

Observation

- The output is directly proportional to the input; therefore, the system is linear.
- A time shift in the input results in the same time shift in the output; hence, the system is time-invariant.
- The output depends only on the present value of the input, making the system causal.

Module Conclusion

In this module, the fundamental concepts of signals and systems were studied using Python. Continuous-time and discrete-time signals were generated and classified based on their characteristics. Various signal operations such as amplitude scaling, time shifting, and time reversal were performed. Elementary signals including the unit step, ramp, impulse, sine, and cosine signals were visualized. Finally, important system properties such as linearity, time

invariance, and causality were verified through graphical analysis. These concepts provide the foundation for advanced topics in Signals and Systems.